This approach offers several advantages:

- *Improved performance*—By rendering components on the server, we can reduce the amount of JavaScript sent to the client, leading to faster initial page loads. The client receives the components in a special format that React knows how to render on the fly.
- *Enhanced SEO*—Server-rendered content is more easily indexable by search engines.
- *Better security*—Sensitive operations can be performed on the server, which prevents exposure of API keys and secrets to the client, reducing the attack surface and ensuring that critical logic and data remain protected.
- *Simplified data fetching*—Data can be fetched on the server, eliminating the need for client-side data-fetching logic.

How does this correlate with our AI-powered solution? It turns out that the Vercel AI SDK offers specialized support for RSCs by providing a set of functions that allows us to combine the best of both worlds. By integrating RSCs with AI capabilities, we gain several significant benefits:

- *Real-time AI-generated UI*—RSCs allows us to stream AI-generated content directly from the server to the client, enabling real-time updates without full page reloads. Streaming happens in the background by the framework seamlessly, and we don't have to deal with technical details on how to do it.
- *Reduced client-side bundles*—Complex AI operations can be performed on the server, reducing the computational load on the client device.
- *Enhanced privacy*—Sensitive user data can be processed on the server, with only the results sent to the client.
- *Improved responsiveness*—By offloading AI processing to the server, the client remains responsive even during complex AI operations.
- *Server actions*—RSCs introduce server actions, allowing us to trigger server-side operations without dealing with explicit API endpoints, complex data management, or unnecessary network roundtrips. This feature simplifies our codebase and reduces the potential for errors in client–server communication.

In fact, adopting RSCs with the Vercel AI SDK often results in reducing the overall lines of code rather than increasing them. This reduction mainly comes from eliminating the need for separate API routes for AI operations, removing boilerplate code for client–server communication, and simplifying state management.

In the following sections, we'll dive into practical examples that demonstrate how to refactor existing AI-powered components to use RSCs. You'll see firsthand how this approach can streamline your codebase while enhancing the capabilities and performance of your AI applications.